

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: ITA 06

COURSE NAME: MACHINE LEARNING

COURSE OUTCOME COVERED

CO5: *Introduction – NumPy Arrays – NumPy Basics- Math – Random – Indexing – Filtering – Statistics – Aggregation – saving data – Data analysis with Pandas – DataFrame – Combining – Indexing – File I/O – Grouping – Filtering – Sorting – Plotting*
(BL 3)

Assessment Tool Weightage: 20%

QUESTIONS:15

Total Marks:25

Annexure A

Pseudocode Writing Task (Algorithm Design)

Code of Conduct:

*I Moshitha Kolanjiyappan (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student: Moshitha k

Annexure A

Short Answer Test

1. Design a pseudocode algorithm to read a dataset from a file and preprocess it using array and DataFrame operations. The algorithm should handle missing values, remove duplicate records, normalize numerical columns, and standardize categorical values. Include steps for indexing, filtering invalid entries, and saving the cleaned dataset. Ensure the algorithm clearly defines input, processing steps, and output while maintaining computational efficiency.
2. Write a detailed pseudocode algorithm to compute statistical measures such as mean, median, standard deviation, and variance from a large dataset. The algorithm should demonstrate how to efficiently use array-based operations, apply filtering to exclude unwanted values, and perform aggregation. Include steps for validating the dataset and handling edge cases such as empty arrays or extreme outliers.
3. Develop a pseudocode algorithm to combine two or more DataFrames containing related data. The algorithm must include steps for aligning indices, merging datasets based on a common key, removing duplicates, and validating the integrity of the merged data. Incorporate grouping and sorting operations, and ensure that constraints such as consistency and completeness of records are maintained.
4. Create a pseudocode algorithm that filters a dataset based on multiple conditions (e.g., value ranges, categories) and then sorts the filtered results. The algorithm should utilize indexing techniques, logical conditions, and efficient sorting methods. Include steps for handling large datasets and optimizing performance while ensuring that the output meets all filtering constraints.
5. Write a pseudocode algorithm for a complete data analysis workflow that includes reading data from files, converting it into DataFrames, performing grouping and aggregation, and generating plots. The algorithm should specify steps for handling file input/output errors, applying transformations, and producing summarized results suitable for visualization. Ensure clarity in each stage from data loading to final output generation.
6. Design a pseudocode algorithm to perform **data cleaning and transformation on time-series data**. The algorithm should handle missing timestamps, interpolate missing values, remove outliers, and resample data at fixed intervals. Include steps for indexing by time, filtering invalid entries, and ensuring chronological consistency while maintaining efficiency.
7. Write a detailed pseudocode algorithm to implement **feature selection** on a dataset. The algorithm should compute correlation between features, remove redundant or irrelevant attributes, and retain significant features. Include steps for normalization, threshold-

based filtering, and validation to ensure improved model performance and reduced dimensionality.

8. Develop a pseudocode algorithm to perform **data partitioning for machine learning**, splitting a dataset into training, validation, and testing sets. Ensure randomization, stratification (if required), and balanced class distribution. Include steps for indexing, shuffling, and validating that no data leakage occurs between partitions.
9. Create a pseudocode algorithm to detect and handle **outliers in a dataset** using statistical methods such as Z-score or IQR. The algorithm should identify extreme values, decide whether to remove or cap them, and ensure that the resulting dataset maintains consistency and reliability for further analysis.
10. Write a pseudocode algorithm to implement **data aggregation and summarization** across multiple categories. The algorithm should group data based on specified attributes, compute aggregated metrics (sum, average, count), and generate a summarized report. Include steps for sorting, indexing, and validating the correctness of aggregated results.
11. Design a pseudocode algorithm to perform **data normalization and scaling** on a dataset. The algorithm should support techniques such as Min-Max scaling and Z-score normalization. Include steps for identifying numerical columns, applying transformations, handling missing values, and validating that all features are scaled correctly within specified ranges.
12. Write a pseudocode algorithm to implement **dimensionality reduction using Principal Component Analysis (PCA)**. The algorithm should include steps for standardizing the dataset, computing covariance matrix, extracting eigenvalues and eigenvectors, and selecting principal components. Ensure efficient matrix operations and validation of reduced dimensions.
13. Develop a pseudocode algorithm to perform **data merging and conflict resolution** from multiple sources. The algorithm should detect conflicting attribute values, apply resolution strategies (e.g., priority rules or averaging), and ensure consistency across the final dataset. Include steps for indexing, validation, and handling missing or inconsistent records.
14. Create a pseudocode algorithm to implement **rolling window computations** on a time-series dataset. The algorithm should calculate moving average and rolling variance using efficient indexing and windowing techniques. Include steps for handling boundary conditions, missing values, and optimizing performance for large datasets.
15. Write a pseudocode algorithm to perform **data validation and integrity checks** on a dataset before analysis. The algorithm should verify data types, detect missing or invalid entries, enforce constraints (such as value ranges), and generate a validation report. Include steps for filtering errors, logging issues, and ensuring the dataset is ready for further processing.

1. Dataset Preprocessing and Cleaning

A dataset preprocessing pipeline should begin by reading the input CSV file into a Pandas DataFrame. The input should be checked to ensure that the file exists and contains valid records. The structure of the DataFrame can be examined using its dimensions, column names, data types, and missing-value counts.

First, numerical and categorical columns are identified. Missing numerical values can be replaced using the median of the corresponding column, while missing categorical values can be replaced with the mode or a value such as “Unknown.” This prevents unnecessary deletion of valid records.

Duplicate records are identified using Pandas duplicate-detection operations and removed when they represent exact duplicates. Numerical columns are then checked against predefined valid ranges. Invalid records can be filtered using Boolean indexing. For example, values such as negative age or impossible medical measurements can be excluded.

Categorical values are standardized by removing unnecessary spaces, converting values to a common case, and mapping equivalent values to a single category. For example, “Male,” “male,” and “M” can all be standardized to “Male.”

Numerical columns can be normalized using Min-Max normalization:

$$\text{Normalized Value} = (\text{Value} - \text{Minimum}) / (\text{Maximum} - \text{Minimum})$$

Indexing and filtering are used to select required rows and columns efficiently. Vectorized Pandas and NumPy operations should be preferred over row-by-row loops to improve computational efficiency.

Finally, the cleaned DataFrame is validated to ensure that missing values and unwanted duplicates have been handled and that all numerical and categorical constraints are satisfied. The cleaned dataset is then saved to a new CSV file.

Pseudocode:

```
BEGIN
  Read dataset from CSV file
  Check file existence and dataset size
  Display columns, data types and missing values

  Identify numerical and categorical columns

  FOR each numerical column
    Replace missing values with median
  END FOR

  FOR each categorical column
    Replace missing values with mode or "Unknown"
    Remove leading/trailing spaces
    Convert values to standard format
  END FOR

  Remove duplicate records
```

```
FOR each numerical column
  Filter values outside valid range
END FOR

Normalize numerical columns

Reset DataFrame index

Validate missing values
Validate duplicate records
Validate numerical ranges
Validate categorical values

Save cleaned dataset to output CSV file
END
```

2. Statistical Measures on a Large Dataset

Statistical analysis of a large dataset can be performed efficiently using NumPy and Pandas aggregation functions. The dataset should first be loaded into a suitable array or DataFrame and checked for missing, infinite, or invalid values.

The first step is to validate the dataset. If the array is empty, statistical calculations should not be performed. Missing values can be removed or handled according to the requirements of the analysis. Filtering can be used to exclude unwanted values before calculating statistics.

The mean is calculated by dividing the sum of all valid observations by the number of observations. The median represents the middle value after sorting. Variance measures how far observations are distributed around the mean, while standard deviation is the square root of variance.

Extreme outliers should be investigated because they can significantly affect the mean and variance. The Interquartile Range method can be used to identify extreme observations. Q1 and Q3 are calculated, followed by:

$$\text{IQR} = \text{Q3} - \text{Q1}$$

The lower and upper boundaries are:

$$\text{Lower Boundary} = \text{Q1} - 1.5 \times \text{IQR}$$

$$\text{Upper Boundary} = \text{Q3} + 1.5 \times \text{IQR}$$

Values outside these boundaries can be flagged for further investigation.

For large datasets, NumPy vectorized functions such as `mean()`, `median()`, `std()`, and `var()` should be used. If the dataset is too large for memory, it can be processed in smaller chunks.

Pseudocode:

```

BEGIN
  Load numerical dataset into array A

  IF A is empty
    Display "Dataset is empty"
    STOP
  END IF

  Remove or filter NaN and infinite values

  IF no valid values remain
    Display "No valid observations"
    STOP
  END IF

  Calculate Q1 and Q3
  Calculate IQR

  Identify extreme outliers

  Filter or retain outliers according to analysis requirements

  Calculate Mean
  Calculate Median
  Calculate Variance
  Calculate Standard Deviation

  Validate number of observations

  Compare results with expected ranges

  Display statistical results
END

```

Vectorized aggregation reduces execution time because operations are performed directly on entire arrays rather than using slow Python loops.

3. Combining Multiple DataFrames

Combining related DataFrames requires careful handling of common keys, indexes, duplicates, and missing records. The first step is to load all DataFrames and identify a common key such as Customer ID, Employee ID, or Product ID.

The key columns should have the same data type and format in all DataFrames. Duplicate keys should be identified using grouping and counting. If a DataFrame is expected to contain one record per customer, more than one record for a customer indicates a potential integrity problem.

The DataFrames can then be merged using the common key. The type of merge depends on the objective. An inner merge retains records common to all datasets, while a left or outer merge can be used when preservation of records is more important.

After merging, grouping by the common key helps identify unexpected duplicates. Sorting the final DataFrame by the key provides consistent ordering, and resetting the index creates a clean sequential index.

The merged DataFrame should be validated by comparing row counts, checking missing keys, detecting duplicate keys, and ensuring required columns are present.

Pseudocode:

```
BEGIN
  Load DataFrame D1
  Load DataFrame D2

  Identify common key

  Check data type of common key
  Convert keys to consistent format

  Group each DataFrame by common key
  Identify duplicate keys

  Remove or resolve unwanted duplicates

  Align required columns

  Merge D1 and D2 using common key

  Identify unmatched records

  Group merged DataFrame by common key
  Check for unexpected duplicates

  Sort merged DataFrame by common key
  Reset index

  Validate completeness and consistency

  Save final merged DataFrame
END
```

This approach maintains data integrity while making it possible to identify mismatched or missing records.

4. Filtering and Sorting Dataset

Filtering is used to select only records that satisfy specific conditions. Multiple conditions can involve numerical ranges, categorical values, dates, or combinations of these conditions.

For example, a dataset may be filtered to select patients whose age is between 18 and 60, whose category is “Active,” and whose measurement is greater than a specified threshold.

Boolean indexing can efficiently apply these conditions. Logical operators such as AND and OR are used to combine conditions. After filtering, the resulting records can be sorted according to one or more columns.

For very large datasets, vectorized filtering should be used because it avoids unnecessary Python loops. Only the required columns should be selected to reduce memory usage.

Pseudocode:

```
BEGIN
  Load dataset D

  Identify required columns

  Define valid numerical range
  Define valid categories
  Define other required conditions

  Create Boolean condition

  Filter dataset using Boolean indexing

  Remove invalid or missing records if required

  Sort filtered records by selected column

  Choose ascending or descending order

  Reset index

  Validate every remaining record

  Save or display filtered and sorted dataset
END
```

The final dataset should contain only records satisfying all specified conditions.

5. Complete Data Analysis Workflow

A complete data analysis workflow consists of data loading, preprocessing, transformation, aggregation, visualization, and reporting.

The first stage is file input. Files should be checked for existence and format before being loaded. CSV files can be read using Pandas. If several files are available, each file can be loaded and combined after checking that their schemas are compatible.

The DataFrames should then be inspected for missing values, duplicate records, inconsistent data types, and invalid values. Required transformations are applied using Pandas and NumPy.

After cleaning, data can be grouped according to relevant categories. Aggregation functions such as sum, mean, count, minimum, and maximum can be used to create summary information.

The summarized DataFrame can then be sorted and used for visualization. Bar charts can represent category comparisons, line charts can represent trends over time, histograms can show distributions, and scatter plots can show relationships between numerical variables.

Errors during file loading should be handled using appropriate error-handling procedures. Corrupted files or files with incompatible structures should be logged rather than causing the entire process to fail.

Pseudocode:

```
BEGIN
  Identify input files

  FOR each input file
    Check whether file exists
    Attempt to read file

    IF file reading fails
      Record error
      Continue with next file
    END IF

    Convert file into DataFrame
  END FOR

  Check DataFrame structure
  Standardize column names
  Standardize data types

  Handle missing and duplicate records

  Apply required transformations

  Group data by selected attributes

  Calculate SUM, COUNT, MEAN, MIN and MAX

  Sort summarized results

  Generate suitable plots

  Add titles, labels and legends

  Save plots and summary results

  Generate final report
END
```

This workflow provides a systematic approach from raw files to meaningful analytical results.

6. Time-Series Data Cleaning and Transformation

Time-series data must be processed carefully because the order and timing of observations are important. The first step is to load the data and convert the timestamp column into a proper datetime format.

Invalid timestamps should be removed or corrected when possible. The data should then be sorted chronologically, and the timestamp should be used as the DataFrame index.

Missing timestamps can be identified by creating a complete time index at the required frequency. The DataFrame can then be reindexed against this complete timeline.

Missing measurement values can be estimated using interpolation when appropriate. For continuous sensor or measurement data, linear interpolation is commonly used.

Outliers can be detected using statistical techniques such as the IQR method or Z-score. Depending on the application, outliers may be removed or capped.

Finally, the time series can be resampled into fixed intervals such as minutes, hours, or days. Aggregation functions such as mean, sum, minimum, and maximum can be applied.

Pseudocode:

BEGIN

 Load time-series dataset

 Convert timestamp column to datetime

 Remove invalid timestamps

 Sort records by timestamp

 Set timestamp as index

 Identify duplicate timestamps

 Create complete time index

 Reindex dataset

 Identify missing measurements

 Interpolate missing values

 Calculate outlier boundaries

 Identify outliers

 Remove or cap outliers

 Resample data at required frequency

 Apply appropriate aggregation

 Verify chronological order

Check remaining missing values

Save cleaned time-series dataset
END

Efficient Pandas time-series functions should be used instead of manually processing every timestamp.

7. Feature Selection

Feature selection reduces the number of input attributes used by a machine-learning model. It improves computational efficiency, reduces memory requirements, and may improve model performance.

The first step is to identify numerical and categorical features and separate the target variable. Missing values should be handled before calculating correlations.

Numerical features should be normalized when necessary. A correlation matrix can then be calculated to identify highly related features. If two features have a correlation greater than a predefined threshold, one may be removed because it contains redundant information.

Features with extremely low variance, excessive missing values, or no meaningful relationship with the target may also be removed.

After feature selection, the reduced dataset should be validated and compared with the original feature set. A machine-learning model can be trained using both versions to determine whether performance has improved or remained acceptable.

Pseudocode:

```
BEGIN
  Load dataset

  Separate target variable

  Identify input features

  Handle missing values

  Normalize numerical features

  Calculate correlation matrix

  Set correlation threshold

  FOR each pair of features
    IF absolute correlation > threshold
      Mark one feature as redundant
    END IF
  END FOR
```

```
Remove redundant features
Remove low-variance features
Remove features with excessive missing values
Create reduced feature dataset
Validate number of selected features
Train model using selected features
Compare model performance
Return significant features
END
```

Feature selection should be performed carefully to ensure that useful information is not accidentally removed.

8. Data Partitioning for Machine Learning

Data partitioning divides a dataset into training, validation, and testing subsets. A common distribution is 70% training, 15% validation, and 15% testing, although the exact proportions may vary.

Before splitting, the dataset should be cleaned and checked for duplicate records. The features and target variable should be separated.

Randomization prevents the order of the original dataset from influencing the partitions. A fixed random seed should be used when reproducible results are required.

For classification problems, stratified splitting should be used to maintain approximately the same class distribution in each partition.

Data leakage must be prevented. The same record must not appear in multiple partitions. For datasets involving patients, customers, or other entities, all records belonging to the same entity may need to remain in one partition.

Normalization and other preprocessing parameters should be calculated using only the training set and then applied to validation and test sets.

Pseudocode:

```
BEGIN
  Load dataset

  Separate features X and target Y

  Validate dataset
```

Remove duplicate records

Define training, validation and testing ratios

Shuffle dataset using random seed

IF classification problem
 Perform stratified splitting
ELSE
 Perform random splitting
END IF

Create training set
Create validation set
Create testing set

Check class distribution

Check duplicate records across partitions

Check entity overlap

Ensure preprocessing uses training data only

Validate final partition sizes

END

This process provides reliable evaluation and prevents data leakage.

9. Outlier Detection and Handling

Outliers are observations that are significantly different from the majority of the dataset. They may represent measurement errors, data-entry problems, or genuine extreme observations.

Two common methods are the Z-score method and the IQR method.

For the Z-score method:

$$\mathbf{Z = (X - Mean) / Standard Deviation}$$

Values whose absolute Z-score exceeds a selected threshold, commonly 3, can be considered potential outliers.

The IQR method calculates Q1 and Q3 and determines:

$$\mathbf{IQR = Q3 - Q1}$$

The lower and upper limits are:

$$\mathbf{Lower = Q1 - 1.5 \times IQR}$$

$$\text{Upper} = Q3 + 1.5 \times \text{IQR}$$

Values outside these limits are flagged.

After identifying outliers, the analyst must decide whether to remove them, cap them at a boundary, or retain them. The decision should depend on the purpose of the analysis and whether the values represent valid observations.

Pseudocode:

```
BEGIN
  Load numerical dataset

  Handle missing values

  Select numerical features

  Choose Z-score or IQR method

  IF Z-score method
    Calculate mean
    Calculate standard deviation
    Calculate Z-score
    Mark values beyond threshold
  ELSE
    Calculate Q1 and Q3
    Calculate IQR
    Calculate lower boundary
    Calculate upper boundary
    Mark values outside boundaries
  END IF

  Count outliers

  Decide whether to remove or cap outliers

  Apply selected treatment

  Compare statistics before and after treatment

  Check data consistency

  Save cleaned dataset
END
```

Outlier treatment should preserve legitimate extreme observations whenever possible.

10. Data Aggregation and Summarization

Data aggregation combines many records into meaningful summaries. Pandas GroupBy operations are particularly useful for this purpose.

First, the dataset is loaded and the required grouping attributes are identified. Examples include region, department, product, category, or year.

The data is grouped using the selected attributes. Aggregation functions can then calculate total values, averages, counts, minimums, and maximums.

For example, sales data can be grouped by product and region to calculate total sales and average sales for each group.

The summary DataFrame can be sorted according to an important metric, such as total sales. The index can be reset to create a clean table.

Validation is important. The total of the aggregated values should be compared with the total of the original dataset where appropriate. Unexpected differences can indicate missing records, incorrect filtering, or grouping errors.

Pseudocode:

```
BEGIN
  Load dataset

  Identify grouping attributes

  Validate required columns

  Group dataset by selected attributes

  Calculate:
    Total = SUM
    Average = MEAN
    Count = COUNT
    Minimum = MIN
    Maximum = MAX

  Create summary DataFrame

  Sort summary by selected metric

  Reset index

  Compare aggregated totals with original totals

  Check for missing or unexpected groups

  Generate summary report

  Save report
END
```

Aggregation simplifies large datasets and makes trends easier to interpret.

11. Data Normalization and Scaling

Normalization and scaling transform numerical variables into comparable ranges. This is especially important for machine-learning algorithms that are sensitive to feature magnitude.

Two common techniques are Min-Max scaling and Z-score normalization.

Min-Max scaling is given by:

$$X' = (X - X_{\min}) / (X_{\max} - X_{\min})$$

This generally transforms values into the range 0 to 1.

Z-score normalization is:

$$X' = (X - \mu) / \sigma$$

where μ is the mean and σ is the standard deviation.

The first step is to identify numerical columns and handle missing values. Constant columns must also be identified because their maximum and minimum values are equal.

Categorical variables should not normally be directly scaled. They should be encoded separately when required.

After scaling, the transformed values should be validated. Min-Max features should normally have minimum approximately 0 and maximum approximately 1. Z-score features should have mean approximately 0 and standard deviation approximately 1.

Pseudocode:

BEGIN

 Load dataset

 Identify numerical columns

 Handle missing numerical values

 FOR each numerical column

 IF Min-Max scaling

 Calculate minimum

 Calculate maximum

 IF maximum != minimum

 Apply Min-Max formula

 ELSE

 Mark column as constant

 END IF

 ELSE IF Z-score scaling

 Calculate mean

 Calculate standard deviation


```
        IF standard deviation != 0
            Apply Z-score formula
        ELSE
            Mark column as constant
        END IF
    END IF
END FOR

Keep categorical columns separate

Validate scaled values

Save transformed dataset
END
```

For machine learning, scaling parameters should be learned from the training data to prevent data leakage.

12. Dimensionality Reduction Using PCA

Principal Component Analysis, or PCA, is a dimensionality-reduction technique that transforms a large number of correlated features into a smaller number of uncorrelated principal components.

The first step is to select numerical features and handle missing values. The features are standardized so that variables with larger numerical scales do not dominate the analysis.

The covariance matrix is then calculated. Eigenvalues and eigenvectors of the covariance matrix are determined. Eigenvalues indicate the amount of variance explained by each principal component, while eigenvectors define the directions of the components.

The eigenvalues are sorted in descending order, and the corresponding eigenvectors are reordered in the same way. The first K components are selected based on the required explained variance.

The standardized data is then projected onto the selected eigenvectors. This produces a reduced dataset with fewer dimensions.

Pseudocode:

```
BEGIN
    Load dataset

    Select numerical features

    Handle missing values

    Calculate mean and standard deviation

    Standardize all features

    Calculate covariance matrix
```

```
Calculate eigenvalues and eigenvectors
Sort eigenvalues in descending order
Sort corresponding eigenvectors
Calculate explained variance
Select first K principal components
Create projection matrix
Multiply standardized data by projection matrix
Generate reduced dataset
Validate number of rows and columns
Validate explained variance
```

END

PCA can significantly reduce computational requirements while retaining most of the important information in the original dataset.

13. Data Merging and Conflict Resolution

Data merging combines information from different sources into one consistent dataset. Before merging, column names, data types, units, and formats should be standardized.

A common unique key should be identified, such as Product ID, Customer ID, or Patient ID. The key should be indexed or used as the basis for merging.

After merging, conflicting values must be detected. For example, two sources may provide different prices for the same product. A conflict-resolution strategy should be defined before selecting a value.

Possible strategies include giving priority to the most trusted source, selecting the most recent record, calculating an average for numerical values, or requesting manual verification.

Missing values can be filled from another trusted source when appropriate. Grouping by the common key can detect duplicate records after merging.

Pseudocode:

```
BEGIN
  Load datasets from all sources

  Standardize column names

  Standardize data types and units
```

```
Identify common key

Set or validate index

Merge datasets using common key

Detect conflicting attribute values

FOR each conflict
    Apply source priority rule
    OR select most recent value
    OR calculate appropriate average
END FOR

Handle missing values

Group by common key

Identify duplicate records

Filter inconsistent records

Sort final dataset

Validate completeness and consistency

Save integrated dataset
END
```

A well-defined conflict-resolution strategy prevents arbitrary decisions and improves the reliability of the final dataset.

14. Rolling Window Computations

Rolling-window calculations are commonly used in time-series analysis to calculate moving averages, moving variance, and other statistics over a fixed number of observations.

The dataset should first be loaded and timestamps converted to datetime format. Records must be sorted chronologically and the timestamp should be used as the index.

A window size is selected. For every position in the time series, the observations belonging to the current window are identified. The moving average is calculated as the mean of the observations in that window, while rolling variance measures their variability.

At the beginning of the dataset, a complete window may not be available. The program must therefore decide whether to return missing values or calculate statistics using a smaller minimum window.

Pandas rolling functions can perform these operations efficiently without manually writing loops.

Pseudocode:

```
BEGIN
  Load time-series dataset

  Convert timestamps to datetime

  Sort data chronologically

  Set timestamp as index

  Define window size W

  FOR each observation
    Select current W observations

    IF number of observations < required minimum
      Assign missing value
    ELSE
      Calculate moving average
      Calculate rolling variance
    END IF
  END FOR

  Store results in new columns

  Check boundary values

  Check handling of missing observations

  Validate first few rolling calculations manually

  Use vectorized rolling operations for large datasets

  Save final results
END
```

Efficient rolling functions reduce repeated calculations and are preferable to manually processing every row.

15. Data Validation and Integrity Checks

Data validation ensures that a dataset is accurate, consistent, and suitable for analysis. Validation should be performed before any major analytical or machine-learning operation.

The first step is to verify that the input file exists and can be opened. The number of rows and columns should be checked, along with the presence of all required columns.

The data types of each column should be compared with the expected schema. Missing values, duplicate records, invalid numerical values, and invalid categorical values should be identified.

Constraints should then be enforced. For example, age must fall within a medically or logically valid range, numerical measurements must be within permitted limits, dates must be valid, and categorical fields must belong to an approved set of categories.

Invalid records should be placed in a separate error DataFrame rather than immediately deleted. Each error should be logged with information such as row ID, column name, invalid value, and error type.

After corrections, the validation process should be repeated. The final dataset should satisfy all required constraints.

Pseudocode:

```
BEGIN
  Load dataset

  Check file availability

  Verify number of rows and columns

  Verify required column names

  Check expected data types

  Detect missing values

  Detect duplicate records

  Detect invalid numerical values

  Check categorical values

  Apply range constraints

  Filter invalid records

  Store invalid records in error table

  Log:
    Row ID
    Column
    Invalid value
    Error type

  Correct errors where possible

  Re-run validation

  IF all constraints are satisfied
    Mark dataset as valid
  ELSE
    Report remaining errors
  END IF

  Generate validation report

  Save validated dataset
```

END

A validation report should include the total number of records, valid records, invalid records, missing values, duplicates, and constraint violations. This provides confidence that the dataset is ready for further processing, statistical analysis, visualization, or machine-learning applications.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	25	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	25	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	20	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	15	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	15	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand